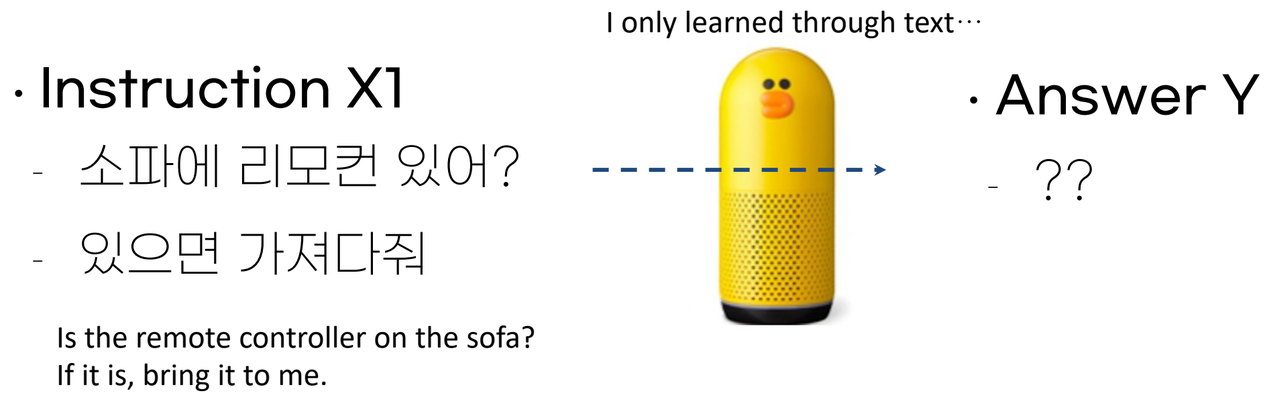
Vision-Language Model

KyungTae Lim

Contents

01 Introduction (background)

- The typical limitation of LLMs
  - LLMs that cannot adapt multimodality* have clear limitations (*gestures, facial expressions, voice, natural language, etc.)



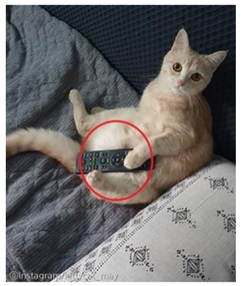
01 Introduction (background)

- The typical limitation of LLMs
  - LLMs that cannot adapt multimodality* have clear limitations (*gestures, facial expressions, voice, natural language, etc.)

Is the remote on the sofa?
If it is, bring it to me.

- Instruction (X1)
  - 소파에 리모컨 있어?
  - 있으면 가져다줘

- Image (X2)



I can now see!

- Answer (Y)
  - 응

Yes.
You servant fool, the master has it.

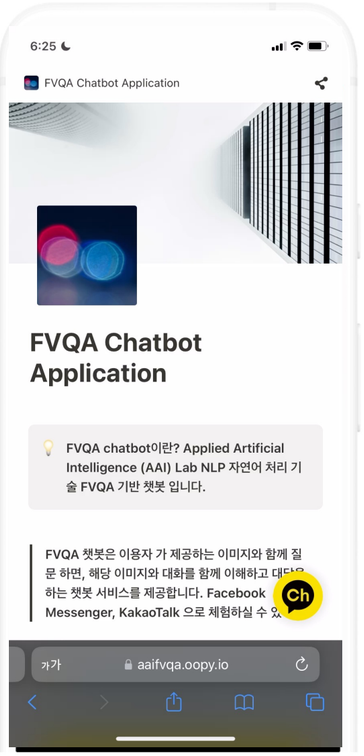
  - 집사야 주인님이 가지고있어

- Impossible to solve without contextual information

--> We need something for the Vision-Language processing module

01 Task: Visual Question Answering (VQA)

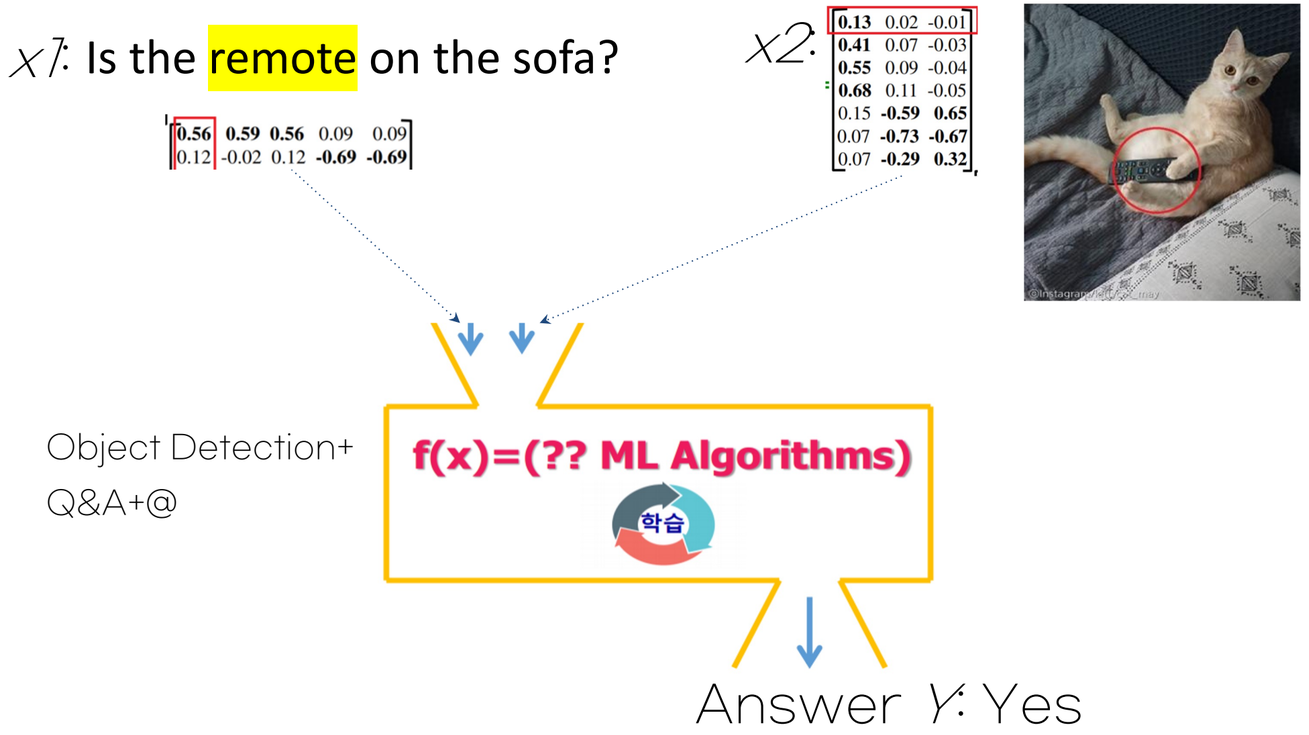
- Visual Question Answering (VQA) is a question-answering task based on understanding contexts between visual and question information.



01. VQA in 2015~8

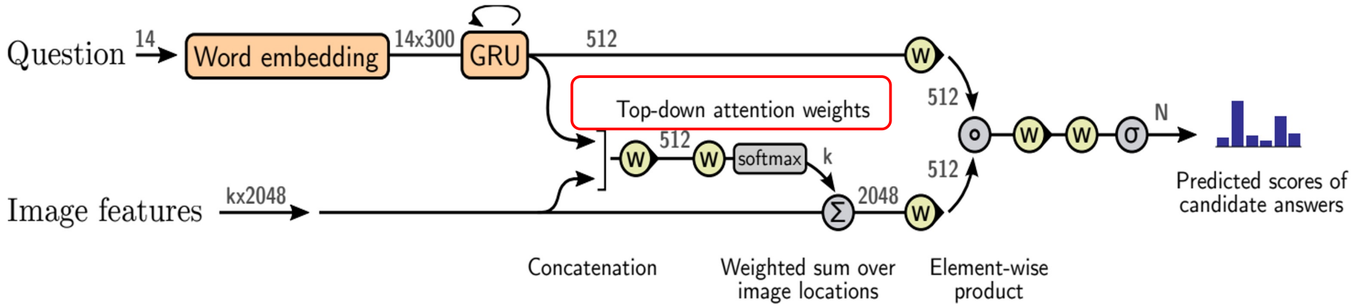
- So how can we perform natural language and image processing simultaneously?
  - Align the modality representations of similar objects (train them with similar representations)

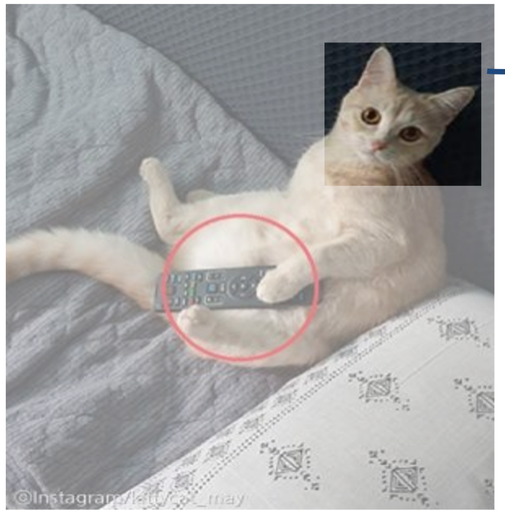
= Train the model to understand that the word "remote control" and the image of a "remote control" represent the same thing.



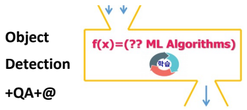
01. VQA in 2015~8

- So how can we perform natural language and image processing simultaneously?
  - Align the modality representations of similar objects (train them with similar representations)→ Attention?



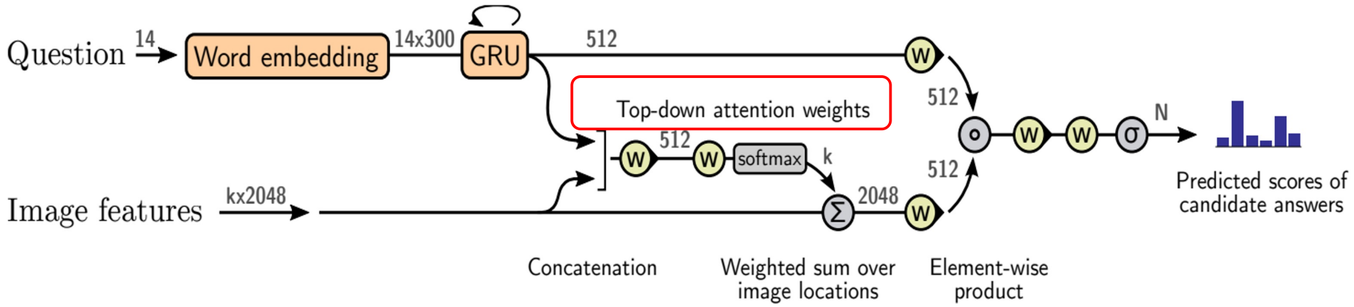


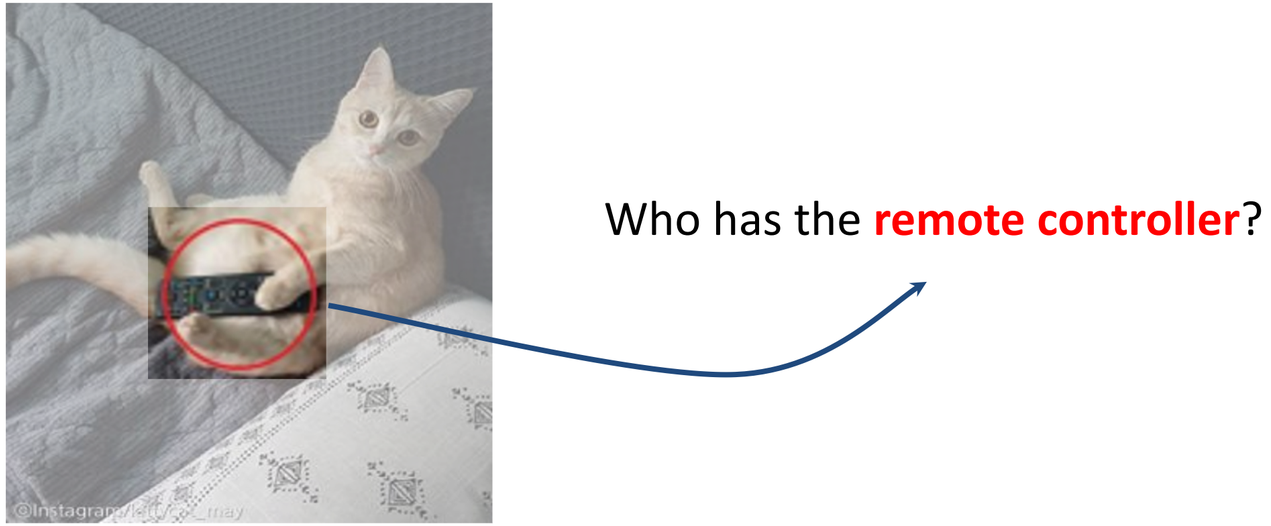
Who has the remote controller?



01. VQA in 2015~8

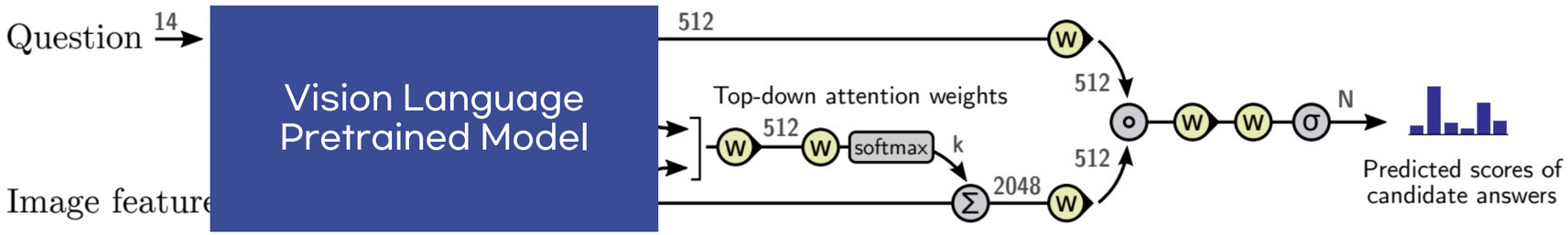
- So how can we perform natural language and image processing simultaneously?
  - Align the modality representations of similar objects (train them with similar representations)→ Attention?

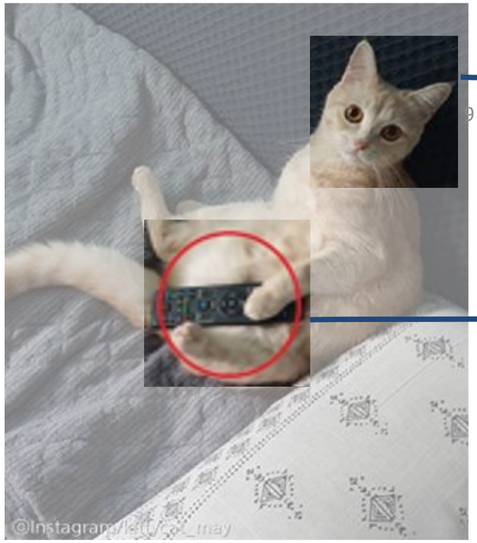




01. VQA in 2018~

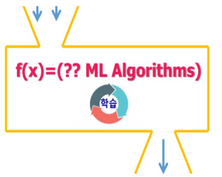
- VQA System with Vision Language Pretrained Model





Who has the remote controller?

Object Detection
+QA+@



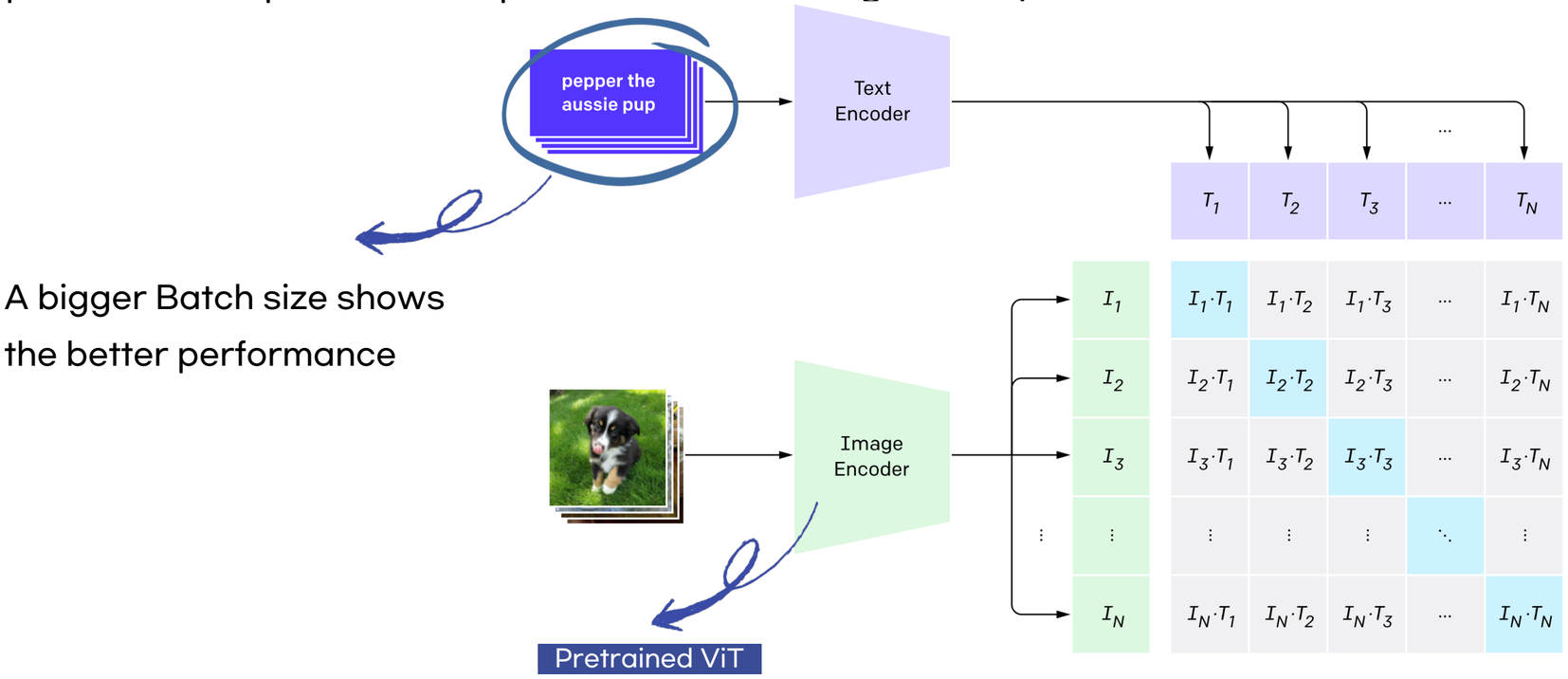
01. CLIP: Contrastive Learning on VLM

- How to train image encoder for VLMs?? CLIP

  - The pretraining of early vision-language models was conducted using Contrastive Learning.

  - This method trains the model to bring related image-caption pairs closer together

    and push unrelated pairs further apart based on the image and caption information.



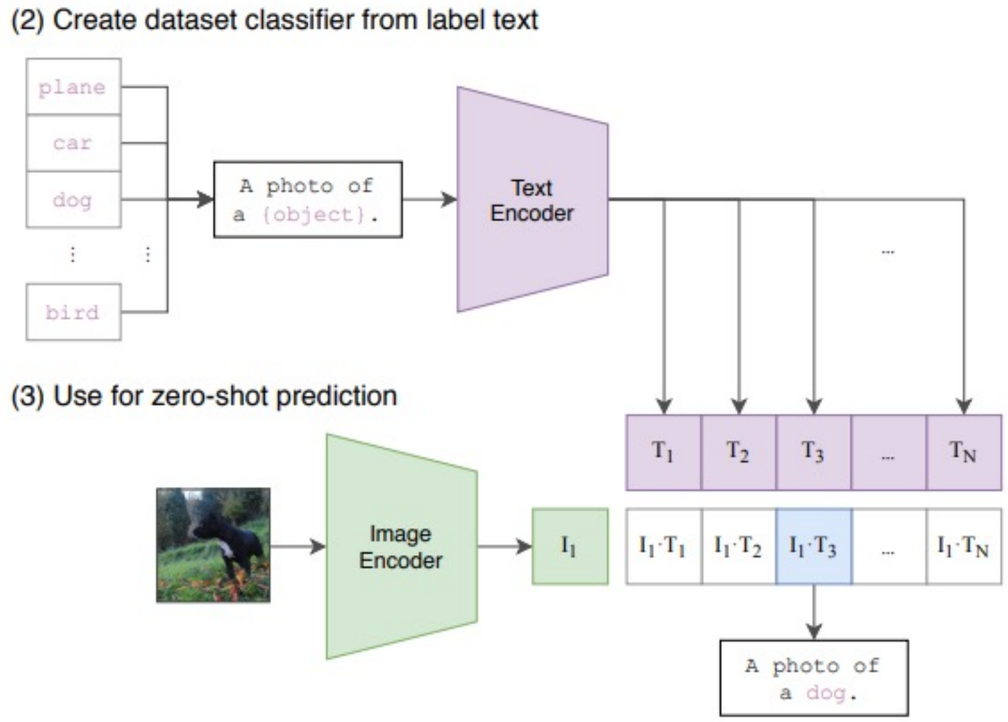
A bigger Batch size shows
the better performance

01. CLIP: Contrastive Learning on VLM

- The pretraining of early vision-language models was conducted using Contrastive Learning.

  - This method trains the model to bring related image-caption pairs closer together

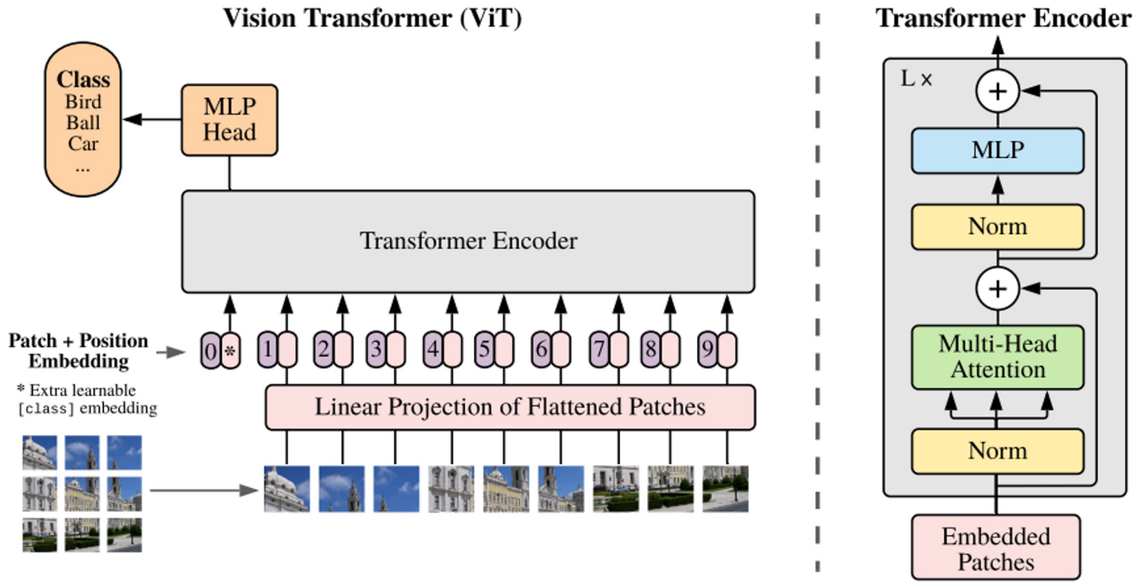
    and push unrelated pairs further apart based on the image and caption information.



01. Vision Transformer (ViT)

- How to train image encoder for VLMs?? Vision Transformer (ViT)
  - ViT splits an image into patches and feeds them as input to a Transformer.
  - It models images, which lack a sequential nature, in a sequential (time-series) format.

- Data
- Patches Embeddings
  - CLS Token
  - Position Embedding
- Transformer
  - Transformer Encoder
  - Attention
  - Residuals
  - MLP
- Head
- ViT



01. Vision Transformer (ViT)

- How to train image encoder for VLMs?? Vision Transformer (ViT)
  - ViT splits an image into patches and feeds them as input to a Transformer.

An overview of the model is depicted in Figure 1. The standard Transformer receives as input a 1D sequence of token embeddings. To handle 2D images, we reshape the image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ into a sequence of flattened 2D patches $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (H, W) is the resolution of the original image, C is the number of channels, (P, P) is the resolution of each image patch, and $N = HW/P^2$ is the resulting number of patches, which also serves as the effective input sequence length for the Transformer. The Transformer uses constant latent vector size D through all of its layers, so we flatten the patches and map to D dimensions with a trainable linear projection (Eq. 1). We refer to the output of this projection as the patch embeddings.

01. Vision Transformer (ViT)

- How to train image encoder for VLMs?? Vision Transformer (ViT)
  - ViT splits an image into patches and feeds them as input to a Transformer.

An overview of the model is depicted in Figure 1. The standard Transformer receives as input a 1D sequence of token embeddings. To handle 2D images, we reshape the image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ into a sequence of flattened 2D patches $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (H, W) is the resolution of the original image, C is the number of channels, (P, P) is the resolution of each image patch, and $N = HW/P^2$ is the resulting number of patches, which also serves as the effective input sequence length for the Transformer. The Transformer uses constant latent vector size D through all of its layers, so we flatten the patches and map to D dimensions with a trainable linear projection (Eq. 1). We refer to the output of this projection as the patch embeddings.

```python
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels: int = 3, patch_size: int = 16, emb_size: int = 768):
        self.patch_size = patch_size
        super().__init__()
        self.projection = nn.Sequential(
            # using a conv layer instead of a linear one -> performance gains
            nn.Conv2d(in_channels, emb_size, kernel_size=patch_size, stride=patch_size),
            Rearrange('b e (h) (w) -> b (h w) e'),
        )

    def forward(self, x: Tensor) -> Tensor:
        x = self.projection(x)
        return x

print("Before Embedding:", x.shape)
print("After Embedding:", PatchEmbedding()(x).shape)
```

- Input: $(N, C_{in}, H_{in}, W_{in})$ or (C_{in}, H_{in}, W_{in})
- Output: $(N, C_{out}, H_{out}, W_{out})$ or $(C_{out}, H_{out}, W_{out})$, where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel\_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel\_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

```
Before Embedding: torch.Size([1, 3, 224, 224])
After Embedding: torch.Size([1, 196, 768])
```

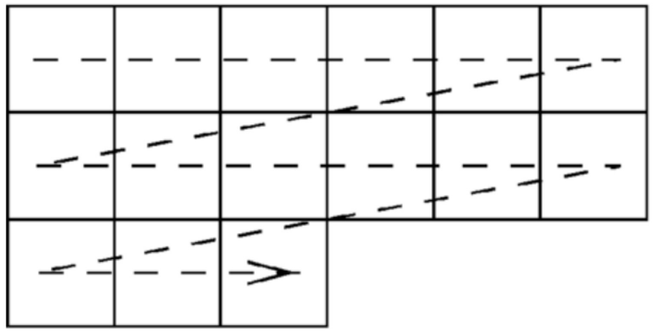
01. Vision Transformer (ViT)

- How to train image encoder for VLMs?? Vision Transformer (ViT)
  - It models images, which lack a sequential nature, in a sequential (time-series) format.

D.4 POSITIONAL EMBEDDING

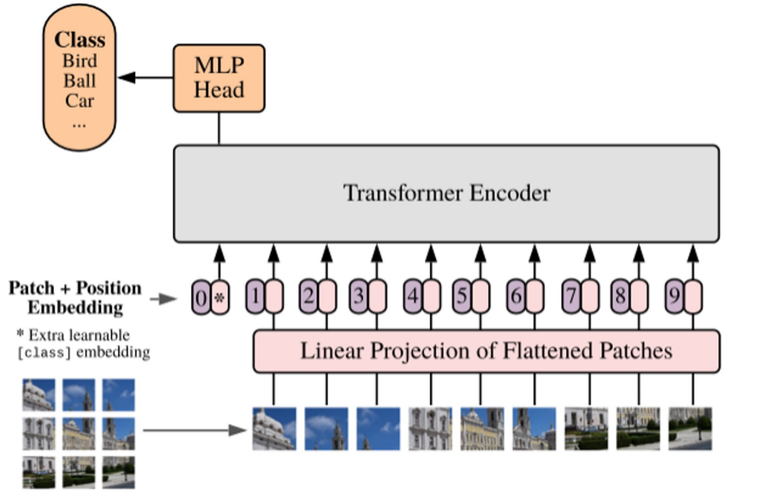
We ran ablations on different ways of encoding spatial information using positional embedding. We tried the following cases:

- Providing no positional information: Considering the inputs as a *bag of patches*.
- 1-dimensional positional embedding: Considering the inputs as a sequence of patches in the raster order (default across all other experiments in this paper).
- 2-dimensional positional embedding: Considering the inputs as a grid of patches in two dimensions. In this case, two sets of embeddings are learned, each for one of the axes, X-embedding, and Y-embedding, each with size $D/2$. Then, based on the coordinate on the path in the input, we concatenate the X and Y embedding to get the final positional embedding for that patch.



01. Vision Transformer (ViT)

- How to train image encoder for VLMs?? Vision Transformer (ViT)
  - It models images, which lack a sequential nature, in a sequential (time-series) format.



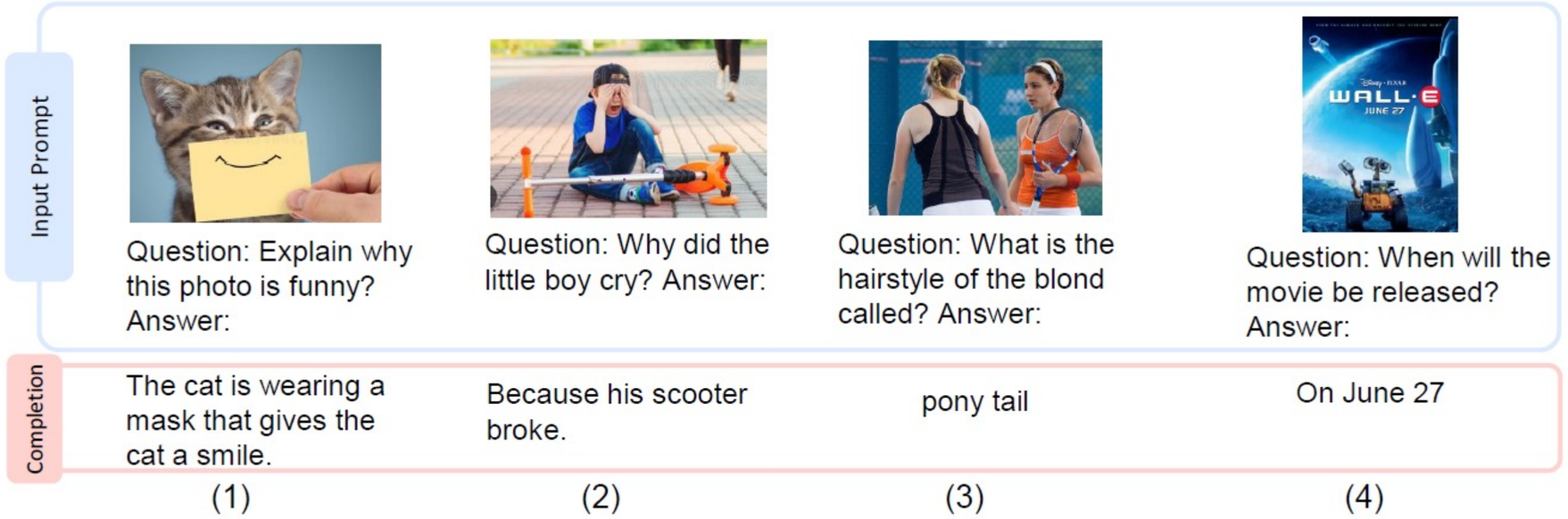

```python
class ViT(nn.Sequential):
    def __init__(self,
                 in_channels: int = 3,
                 patch_size: int = 16,
                 emb_size: int = 768,
                 img_size: int = 224,
                 depth: int = 12,
                 n_classes: int = 1000,
                 **kwargs):
        super().__init__(
            PatchEmbedding(in_channels, patch_size, emb_size, img_size),
            TransformerEncoder(depth, emb_size=emb_size, **kwargs),
            ClassificationHead(emb_size, n_classes)
        )

```

But if you look closely at the input, it only contains images... so this model only processes images, right?
How can we process both images and language together?

01. Introduction of VLM

- Vision-Language Model (VLM)
  - A model that adds image channels to large language models (LLMs) to process image-text data.
  - It can be used for tasks such as Visual Question Answering (VQA) and image-text retrieval.
  - Recently, methods that align pretrained LLMs with image features have been mainly used.



Language Is Not All You Need: Aligning Perception with Language Models (Shaohan et al.)

01. VisualBERT: VLM over BERT

- Another reasonable approach proposed is to input image patches together when training BERT.

  - This method is very simple but significantly improves VQA performance.

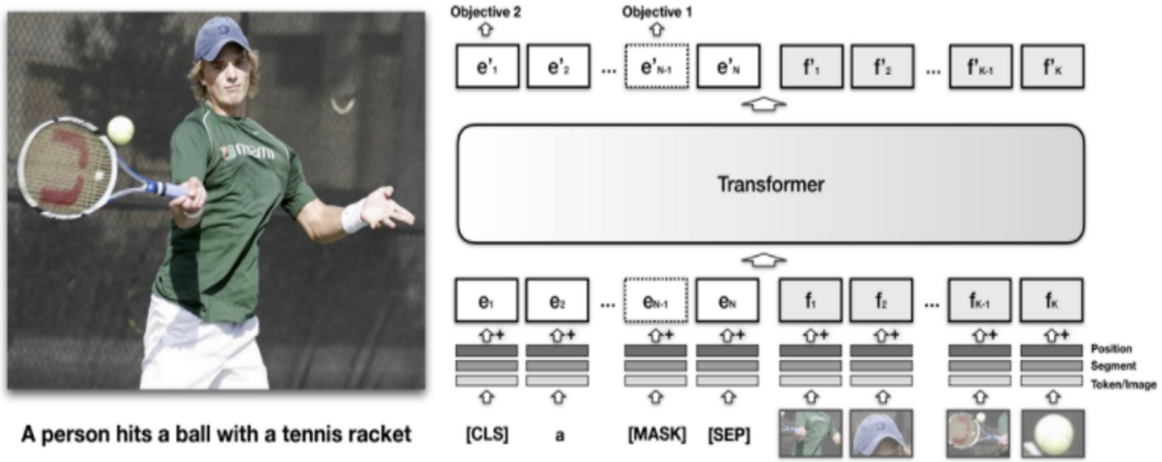


Figure 2: The architecture of VisualBERT. Image regions and language are combined with a Transformer to allow the self-attention to discover implicit alignments between language and vision. It is pre-trained with a masked language modeling (Objective 1), and sentence-image prediction task (Objective 2), on caption data and then fine-tuned for different tasks. See §3.3 for more details.

```
class BertVisualModel(PreTrainedBertModel):
    def __init__(self, config):
        super(BertVisualModel, self).__init__(config)
        self.embeddings = BertEmbeddingsWithVisualEmbedding(config)
        self.encoder = BertEncoder(config)
        self.pooler = BertPooler(config)
        self.bypass_transformer = config.bypass_transformer

        if self.bypass_transformer:
            self.additional_layer = BertLayer(config)

        self.output_attention_weights = config.output_attention_weights

        self.apply(self.init_bert_weights)
```

```
def __init__(self, config):
    super(BertModel, self).__init__(config)
    self.embeddings = BertEmbeddings(config)
    self.encoder = BertEncoder(config)
    self.pooler = BertPooler(config)
    self.apply(self.init_bert_weights)
```

01. Problem statements

- Latest Research Trends in Multimodal Large Language Models

  - Visual BERT requires a large amount of caption data for training… but can’t we just connect pretrained monolingual models?

  - Increased image understanding through visual instruction tuning (LLaVA)
  - Modality feature alignment via Transformer architecture (BLIP-2)

Contents

01. Introduction of LLava

- Pre-training of traditional Vision-Language (VL) models requires a large amount of computation.
- Recently, many high-performance single-modality Vision or Language pre-trained models have been released → This paper proposes a compute-efficient VL pre-training method leveraging these models.

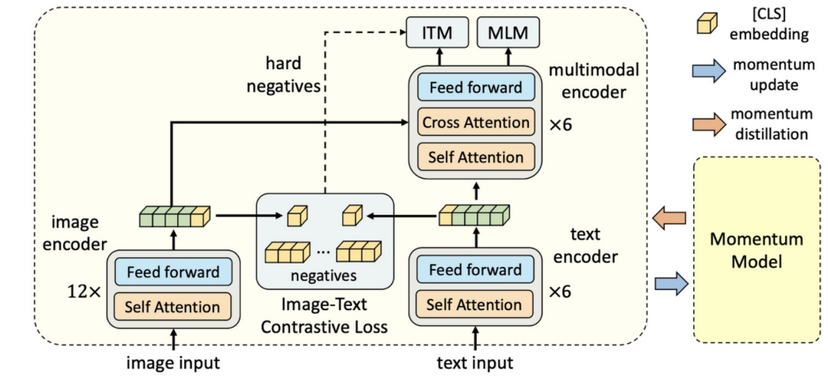
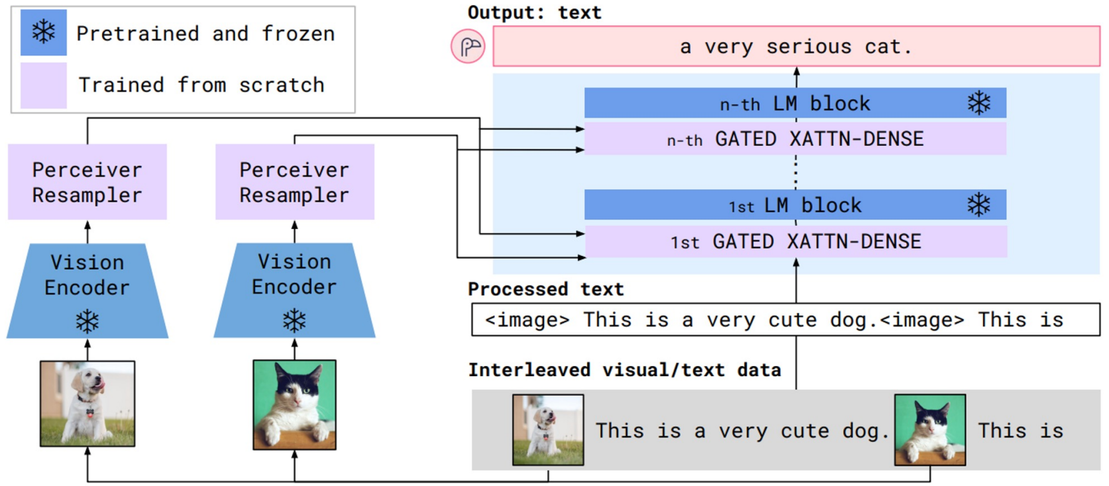


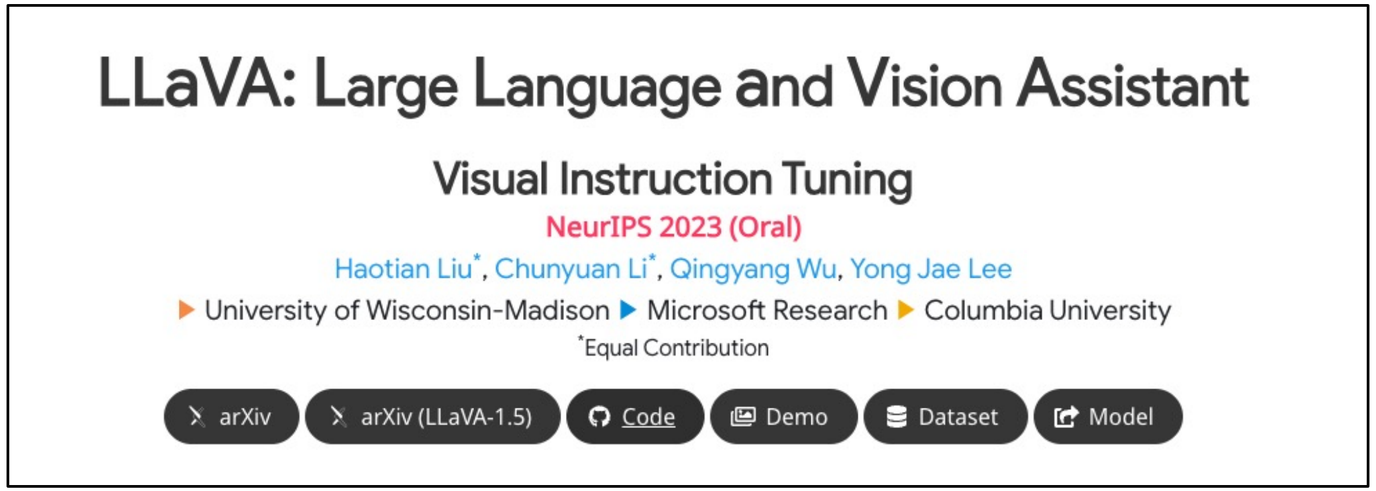
Figure 1: **Illustration of ALBEF.** It consists of an image encoder, a text encoder, and a multimodal encoder. We propose an image-text contrastive loss to align the unimodal representations of an image-text pair before fusion. An image-text matching loss (using in-batch hard negatives mined through contrastive similarity) and a masked-language-modeling loss are applied to learn multimodal interactions between image and text. In order to improve learning with noisy data, we generate pseudo-targets using the momentum model (a moving-average version of the base model) as additional supervision during training.

Traditional VLM (CRL): Albef



Compute-Efficient VL Model: Flamingo

02. LLaVA



- The first study to incorporate instruction tuning in the multimodal field
- Used GPT-4 to generate instruction-following data for training
- Proposed Visual-Instruction Tuning, which extends instruction tuning to the multimodal space

02. LLaVA

2-1. Overview of LLaVA

1. Generating Multimodal Instruction-Following Data

- There is a shortage of visual-language instruction-following data
- Used GPT-4 (text-only) to convert image-text data into instruction-following data

2. Multimodal Large Language Modeling

- Developed a large multimodal model (Multimodal Large Language Model, MLLM)
- LLaVA = CLIP(Vision Encoder) + LLaMA(Large Language Model)

02. LLaVA

- Background : There is a large amount of image-text paired data, but very little instruction-following data.

Generating Visual Instruction-following data

- ### Conversation

  Design a dialogue where an AI assistant and a person ask and answer questions about images.

- ### Detailed description

  To include comprehensive and detailed explanations about images, generate a list of questions and randomly select one question per image from the list to create the description. (This part needs revision.)

- ### Complex reasoning

  Based on the conversation and detailed description, generate deep reasoning questions. The answers require a step-by-step reasoning process.

02. LLaVA

02-02. Instruction-following Data: Examples

Conversation

Question: What type of vehicle is featured in the image?
Answer: The image features a black sport utility vehicle (SUV).
Question: Where is the vehicle parked?
Answer: The vehicle is parked in an underground parking area, likely in a public garage.
Question: What are the people in the image doing?
Answer: The people are trying to fit all of their luggage into the SUV, likely preparing for a trip.

Detailed description

The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip. …

Complex reasoning

Question: What challenges do these people face?
Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases…

02. LLaVA

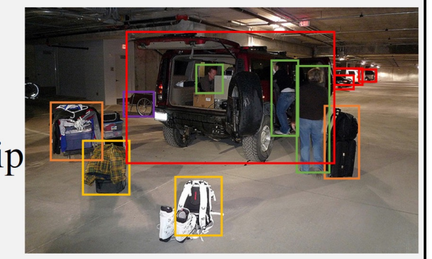
02-02. Instruction-following Data:

- GPT-4 API does not accept image inputs.
- Therefore, captions and bounding boxes, which can provide image information in text form, are utilized.

Caption & BBOX

Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.
Luggage surrounds a vehicle in an underground parking area
People try to fit all of their luggage in an SUV.
The sport utility vehicle is parked in the public garage, being packed for a trip
Some people with luggage near a van that is transporting it.



Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], person: [0.63, 0.222, 0.686, 0.516], person: [0.444, 0.233, 0.487, 0.34], backpack: [0.384, 0.696, 0.485, 0.914], backpack: [0.755, 0.413, 0.846, 0.692], suitcase: [0.758, 0.413, 0.845, 0.69], suitcase: [0.1, 0.497, 0.173, 0.579], bicycle: [0.282, 0.363, 0.327, 0.442], car: [0.786, 0.25, 0.848, 0.322], car: [0.783, 0.27, 0.827, 0.335], car: [0.86, 0.254, 0.891, 0.3], car: [0.261, 0.101, 0.787, 0.626]

02 LLaVA (Data Generation)

Caption & BBOX

Context type 1: Captions
A group of people standing outside of a black vehicle with various luggage.
Luggage surrounds a vehicle in an underground parking area
People try to fit all of their luggage in an SUV.
The sport utility vehicle is parked in the public garage, being packed for a trip
Some people with luggage near a van that is transporting it.

Context type 2: Boxes
person: [0.681, 0.242, 0.774, 0.694], person: [0.63, 0.222, 0.686, 0.516], person: [0.444, 0.233, 0.487, 0.34], backpack: [0.384, 0.696, 0.485, 0.914], backpack: [0.755, 0.413, 0.846, 0.692], suitcase: [0.758, 0.413, 0.845, 0.69], suitcase: [0.1, 0.497, 0.173, 0.579], bicycle: [0.282, 0.363, 0.327, 0.442], car: [0.786, 0.25, 0.848, 0.322], car: [0.783, 0.27, 0.827, 0.335], car: [0.86, 0.254, 0.891, 0.3], car: [0.261, 0.101, 0.787, 0.626]

```
messages = [ {"role":"system", "content": f"""You are an AI visual assistant, and you are seeing a single image. What you see are provided with five sentences, describing the same image you are looking at. Answer all questions as you are seeing the image.

Design a conversation between you and a person asking about this photo. The answers should be in a tone that a visual AI assistant is seeing the image and answering the question. Ask diverse questions and give corresponding answers.

Include questions asking about the visual content of the image, including the object types, counting the objects, object actions, object locations, relative positions between objects, etc. Only include questions that have definite answers:
(1) one can see the content in the image that the question asks about and can answer confidently;
(2) one can determine confidently from the image that it is not in the image. Do not ask any question that cannot be answered confidently.

Also include complex questions that are relevant to the content in the image, for example, asking about background knowledge of the objects in the image, asking to discuss about events happening in the image, etc. Again, do not ask about uncertain details. Provide detailed answers when answering complex questions. For example, give detailed examples or reasoning steps to make the content more convincing and well-organized. You can include multiple paragraphs if necessary."""}
]
for sample in fewshot_samples:
    messages.append({"role":"user", "content":sample['context']})
    messages.append({"role":"assistant", "content":sample['response']} )
messages.append({"role":"user", "content":'\n'.join(query)})
```

Prompt

GPT-4
(text only)

Generated Data

Response type 1: conversation
Question: What type of vehicle is featured in the image?
Answer: The image features a black sport utility vehicle (SUV).
Question: Where is the vehicle parked?
Answer: The vehicle is parked in an underground parking area, likely in a public garage.
Question: What are the people in the image doing?
Answer: The people are trying to fit all of their luggage into the SUV, likely preparing for a trip.

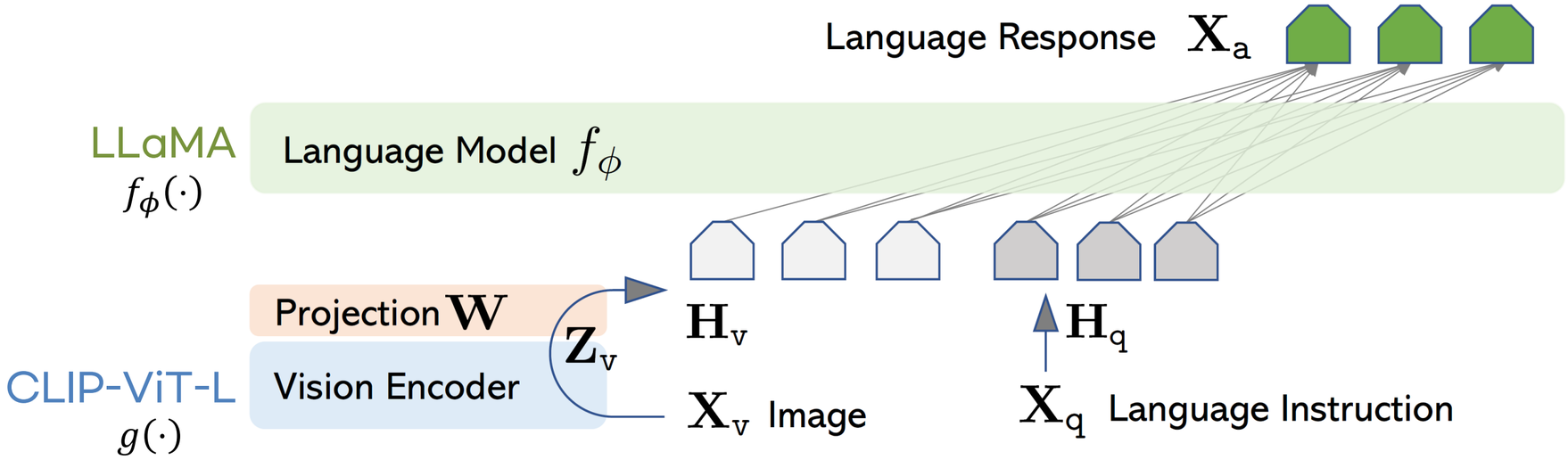
Response type 2: detailed description
The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip.
In the parking area, various luggage items are scattered around the vehicle. There are two backpacks, one located near the left rear wheel and the other closer to the right side of the vehicle. Additionally, there are two suitcases, one on the right side of the car and another further away near the center of the parking area. A bicycle can also be seen on the left side of the vehicle.
Other cars are parked around the main SUV, with one car positioned behind it and slightly to the left, another behind and slightly to the right, and the third car further behind on the right side.

Response type 3: complex reasoning
Question: What challenges do these people face?
Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases and backpacks to be packed, which suggests that the group has a significant amount of belongings to accommodate. They might have to strategize and arrange the luggage efficiently to ensure that everything fits properly into the vehicle. Additionally, they need to consider the comfort of the passengers and visibility while driving, so the placement of the luggage must not obstruct the driver's view or make the passengers uncomfortable during the trip.

Generation with Few-shot samples

02. LLaVA

02-02. Structure of LLaVA



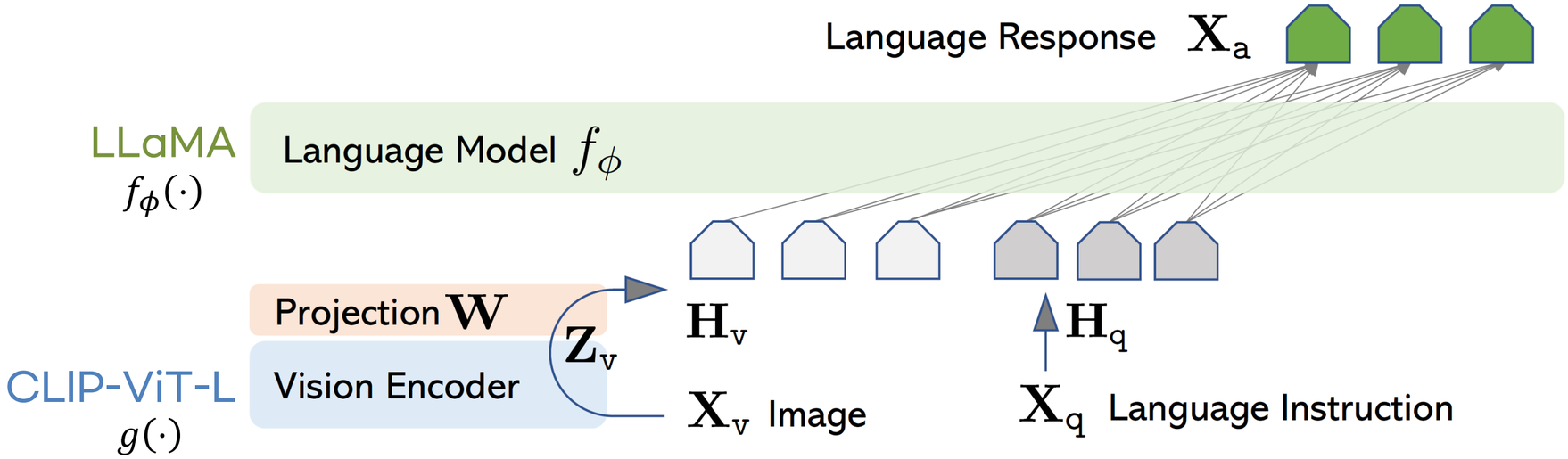
Purpose : Effectively combining pretrained Language Models and Vision Encoders.

Language Model $f_\phi(\cdot)$: LLaMA

Vision Encoder $g(\cdot)$: CLIP (ViT-L/14)

02. LLaVA

02-02. Structure of LLaVA



X_v : Image

X_q : Language Instruction

Z_v : Image representations passed through the visual encoder, $(Z_v = g(X_v))$

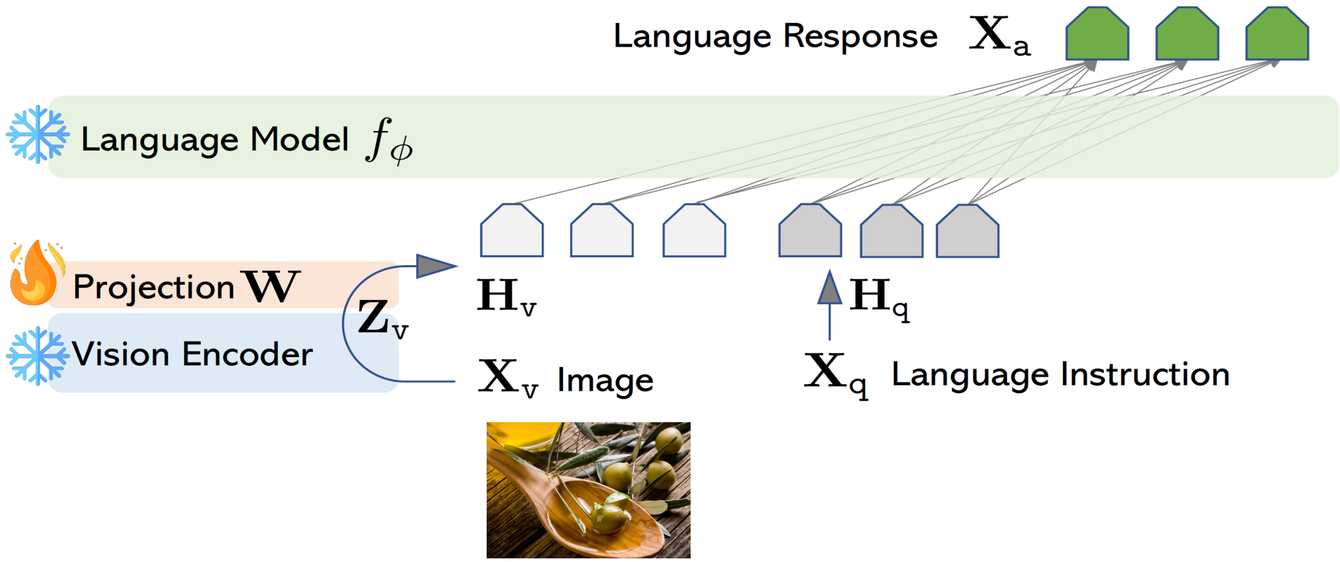
H_v : Convert $\mathbf{Z_v}$ into an embedding format that can be input to the LLM, $(H_v = W \cdot Z_v)$

H_q : Convert $\mathbf{X_q}$ into an embedding format that can be input to the LLM.

02. LLaVA

02-03. Training LLaVA (1-stage)

caption
string
olive oil is a healthy ingredient used liberally .

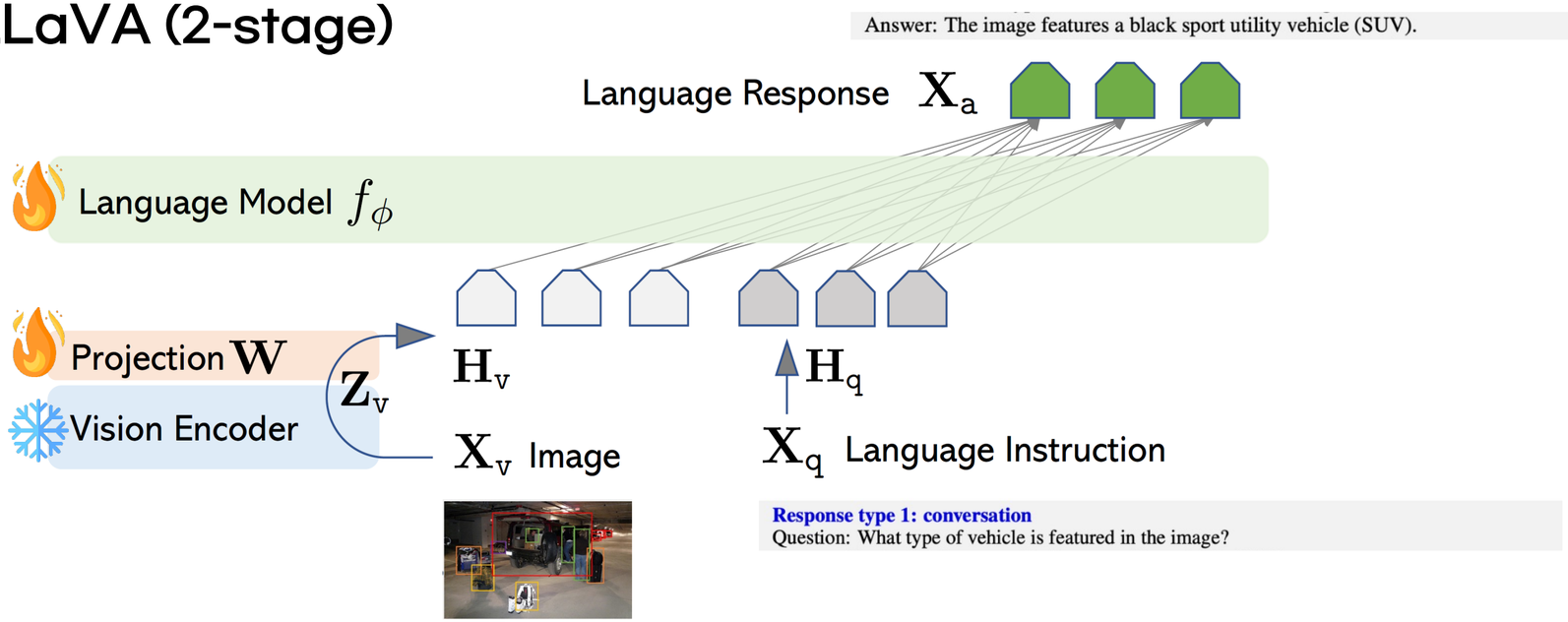


Stage 1 : Pre-training for Feature Alignment

- Using image-caption data (CC3M) to perform feature alignment through single-turn conversations.
  At this time, the weights of the Vision Encoder and Language Model are not updated.
- Only the linear projection weights $\boldsymbol{W}$, which connect the image and language modalities, are updated. → Focus is on feature alignment.

02. LLaVA

02-03. Training LLaVA (2-stage)



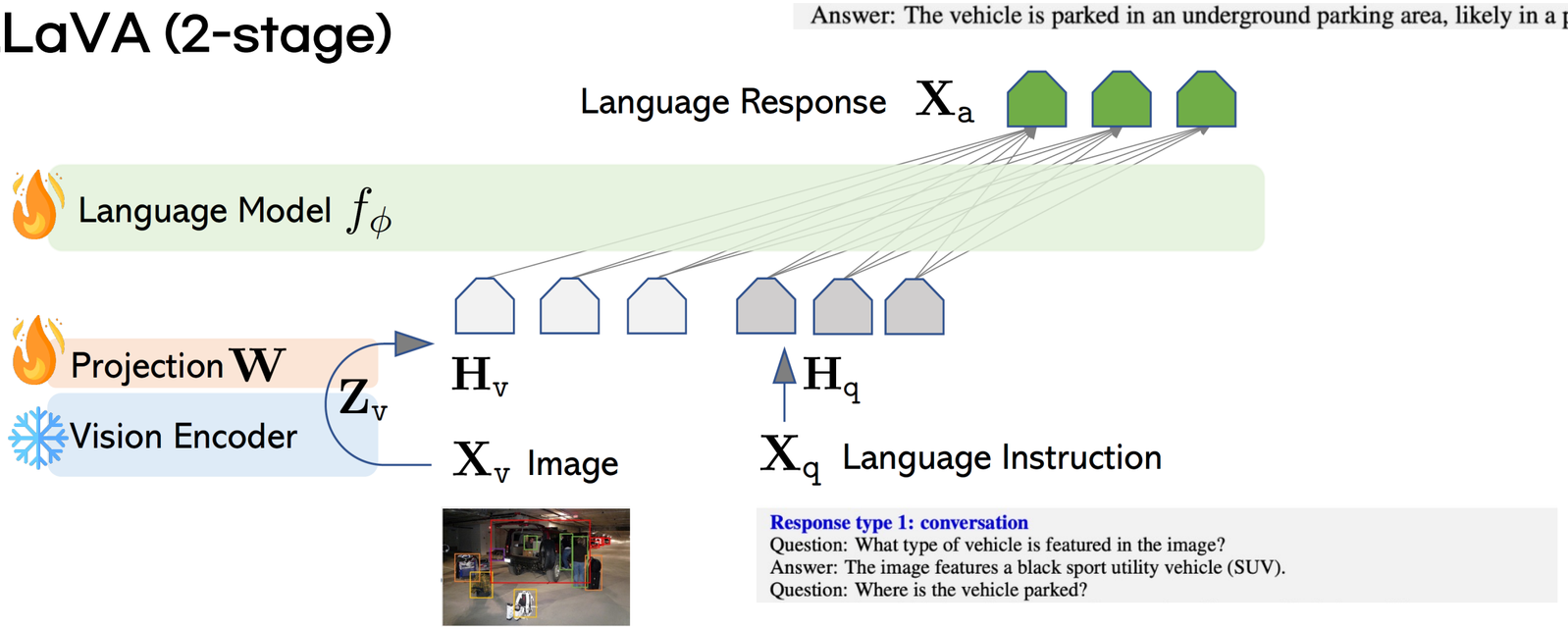
Stage 2 : Fine-tuning End-to-End

- Fine-tuning using the generated instruction-following data (conversation, detailed description, complex reasoning).
- At this stage, the Vision Encoder weights are still not updated; only $\boldsymbol{W}$ and $\boldsymbol{\phi}$(LLM) are updated.

  → Enhances feature alignment and the LLM's image understanding.

02. LLaVA

02-03. Training LLaVA (2-stage)



Stage 2 : Fine-tuning End-to-End

- Fine-tuning using the generated instruction-following data (conversation, detailed description, complex reasoning).
- At this stage, the Vision Encoder weights are still not updated; only $\boldsymbol{W}$ and $\boldsymbol{\phi}$(LLM) are updated.

  → Enhances feature alignment and the LLM's image understanding.

02. LLaVA

02-03. Training LLaVA

1. Generation of Multi-turn conversation (T : total number of turns)

$$(X_q^1, X_a^1, X_q^2, X_a^2, \cdots, X_q^T, X_a^T)$$

2. In the first turn, randomly decide whether to place the image feature before or after the text input (because the user's input order may vary).

$$\mathbf{X}_{\texttt{instruct}}^t = \begin{cases} \text{Random choose } [\mathbf{X}_{\texttt{q}}^1, \mathbf{X}_{\texttt{v}}] \text{ or } [\mathbf{X}_{\texttt{v}}, \mathbf{X}_{\texttt{q}}^1], & \text{the first turn } t = 1 \\ \mathbf{X}_{\texttt{q}}^t, & \text{the remaining turns } t > 1 \end{cases}$$

3. Training Objective (auto-regressive objective)

$$p(\mathbf{X}_{\texttt{a}} | \mathbf{X}_{\texttt{v}}, \mathbf{X}_{\texttt{instruct}}) = \prod_{i=1}^{L} p_{\boldsymbol{\theta}}(\boldsymbol{x}_i | \mathbf{X}_{\texttt{v}}, \mathbf{X}_{\texttt{instruct},<i}, \mathbf{X}_{\texttt{a},<i})$$

All questions up to the previous time step

Current question's answer

All answers up to the previous time step

02. LLaVA

02-03. Training LLaVA

$$\mathbf{X}_{\texttt{system-message}}\ \texttt{<STOP>}\ \backslash\texttt{n}$$
$$\texttt{Human}: \mathbf{X}^1_{\texttt{instruct}}\ \texttt{<STOP>}\ \backslash\texttt{n}\ \texttt{Assistant:}\ \mathbf{X}^1_{\texttt{a}}\ \texttt{<STOP>}\ \backslash\texttt{n}$$
$$\texttt{Human}: \mathbf{X}^2_{\texttt{instruct}}\ \texttt{<STOP>}\ \backslash\texttt{n}\ \texttt{Assistant:}\ \mathbf{X}^2_{\texttt{a}}\ \texttt{<STOP>}\ \backslash\texttt{n}\ \cdots$$

- The model is trained to predict the tokens corresponding to the green highlighted parts (including the <STOP> token).
- Loss is not calculated for the $X_{\text{system}-\text{message}}$ and X_{instruct} parts → Loss is calculated only for the X_a parts

02. LLaVA

02-04. Evaluation of LLaVA

Context type 1: Captions
A group of people standing outside of a black vehicle with various luggage.
Luggage surrounds a vehicle in an underground parking area
People try to fit all of their luggage in an SUV.
The sport utility vehicle is parked in the public garage, being packed for a trip
Some people with luggage near a van that is transporting it.

Context type 2: Boxes
person: [0.681, 0.242, 0.774, 0.694], person: [0.63, 0.222, 0.686, 0.516], person: [0.444, 0.233, 0.487, 0.34], backpack: [0.384, 0.696, 0.485, 0.914], backpack: [0.755, 0.413, 0.846, 0.692], suitcase: [0.758, 0.413, 0.845, 0.69], suitcase: [0.1, 0.497, 0.173, 0.579], bicycle: [0.282, 0.363, 0.327, 0.442], car: [0.786, 0.25, 0.848, 0.322], car: [0.783, 0.27, 0.827, 0.335], car: [0.86, 0.254, 0.891, 0.3], car: [0.261, 0.101, 0.787, 0.626]

```
messages = [ {"role":"system", "content": f"""You are an AI visual assistant, and you are seeing a single image. What you see are provided with five sentences, describing the same image you are looking at. Answer all questions as you are seeing the image.

Design a conversation between you and a person asking about this photo. The answers should be in a tone that a visual AI assistant is seeing the image and answering the question. Ask diverse questions and give corresponding answers.

Include questions asking about the visual content of the image, including the object types, counting the objects, object actions, object locations, relative positions between objects, etc. Only include questions that have definite answers:
(1) one can see the content in the image that the question asks about and can answer confidently;
(2) one can determine confidently from the image that it is not in the image. Do not ask any question that cannot be answered confidently.

Also include complex questions that are relevant to the content in the image, for example, asking about background knowledge of the objects in the image, asking to discuss about events happening in the image, etc. Again, do not ask about uncertain details. Provide detailed answers when answering complex questions. For example, give detailed examples or reasoning steps to make the content more convincing and well-organized. You can include multiple paragraphs if necessary."""}
]
for sample in fewshot_samples:
    messages.append({"role":"user", "content":sample['context']})
    messages.append({"role":"assistant", "content":sample['response']} )
messages.append({"role":"user", "content":'\n'.join(query)})
```

<Caption + BBOX + system message>

LLaVA

Output of LLaVA:
Conv, Description, reasoning

GPT-4 Teacher

GPT-4

Output of GPT-4:
Conv, Description, reasoning

02. LLaVA

02-04. Evaluation of LLaVA

	Conversation	Detail description	Complex reasoning	**All**
Full data	83.1	75.3	96.5	85.1
Detail + Complex	81.5 (-1.6)	73.3 (-2.0)	90.8 (-5.7)	81.9 (-3.2)
Conv + 5% Detail + 10% Complex	81.0 (-2.1)	68.4 (-7.1)	91.5 (-5.0)	80.5 (-4.4)
Conversation	76.5 (-6.6)	59.8 (-16.2)	84.9 (-12.4)	73.8 (-11.3)
No Instruction Tuning	22.0 (-61.1)	24.0 (-51.3)	18.5 (-78.0)	21.5 (-63.6)

- Selected 30 random images from the COCO Val 2014 dataset.
- For each image, generated Conversation, Detailed Description, and Complex Reasoning.
  Used GPT-4 as a teacher to perform quantitative evaluation on a scale of 1 to 10.
- Overall performance improves as the amount of visual instruction tuning data increases.
- LLaVA demonstrates excellent performance with a score of 85.1%.